

Goal2Skill: Long-Horizon Manipulation with Adaptive Planning and Reflection

Zhen Liu^{1,2}, Xinyu Ning^{1,2}, Zhe Hu², XinXin Xie¹, Weize Li¹, Zhipeng Tang², Chongyu Wang², Zejun Yang², Hanlin Wang², Yitong Liu¹, Zhongzhu Pu^{2,3,†}

¹Beijing University of Posts and Telecommunications, ²InspireOmni AI, ³Tsinghua University

Recent vision-language-action (VLA) systems have demonstrated strong capabilities in embodied manipulation. However, most existing VLA policies rely on limited observation windows and end-to-end action prediction, which makes them brittle in long-horizon, memory-dependent tasks with partial observability, occlusions, and multi-stage dependencies. Such tasks require not only precise visuomotor control, but also persistent memory, adaptive task decomposition, and explicit recovery from execution failures. To address these limitations, we propose a dual-system framework for long-horizon embodied manipulation.

Our framework explicitly separates high-level semantic reasoning from low-level motor execution. A high-level *planner*, implemented as a VLM-based agentic module, maintains structured task memory and performs goal decomposition, outcome verification, and error-driven correction. A low-level *executor*, instantiated as a VLA-based visuomotor controller, carries out each sub-task through diffusion-based action generation conditioned on geometry-preserving filtered observations. Together, the two systems form a closed loop between planning and execution, enabling memory-aware reasoning, adaptive replanning, and robust online recovery. Experiments on representative RMBench tasks show that the proposed framework substantially outperforms representative baselines, achieving a 32.4% average success rate compared with 9.8% for the strongest baseline. Ablation studies further confirm the importance of structured memory and closed-loop recovery for long-horizon manipulation.

Date: April 16, 2026

Correspondence: jaycening@inspireomni.ai



1 Introduction

Vision-language-action (VLA) models have recently emerged as a promising paradigm for embodied manipulation, integrating visual perception, language understanding, and action generation into a unified policy (Kawaharazuka et al., 2025; Din et al., 2025; Ma et al., 2024; Jiang et al., 2025). Leveraging large-scale multimodal pretraining, these models demonstrate strong capabilities in instruction following, situational decision making, cross-object generalization, and multi-task control (Yu et al., 2026; Ye et al., 2026; Deng et al., 2025; Team, 2026; Hu et al., 2025). Such advances mark an important step toward scalable embodied intelligence.

Despite this progress, most existing VLA systems are fundamentally designed for *short-horizon* decision making, where actions are generated primarily from recent observations with limited temporal dependency (Hung et al., 2025b; Yue et al., 2024; Xiang et al., 2025). This assumption breaks down in many real-world scenarios. In *long-horizon* manipulation tasks, an agent must maintain persistent task context, decompose high-level goals into interdependent sub-tasks, verify intermediate outcomes, and adapt its behavior when execution deviates from expectations (Huang et al., 2025; Li et al., 2026b; Team et al., 2026; Lian et al., 2026). As illustrated in Figure 1, even a simple task of picking up a Coke can, presenting it to a scanner, and retrying after a failed scan requires sustained memory, intermediate verification, and adaptive correction. These challenges are further exacerbated by occlusions, distractors, constrained interaction spaces, and dynamically evolving scene states. Under such conditions, purely reactive policies may produce actions that are locally plausible yet globally incorrect, leading to failure accumulation over time (Zhang et al., 2026a; Yang et al., 2026).

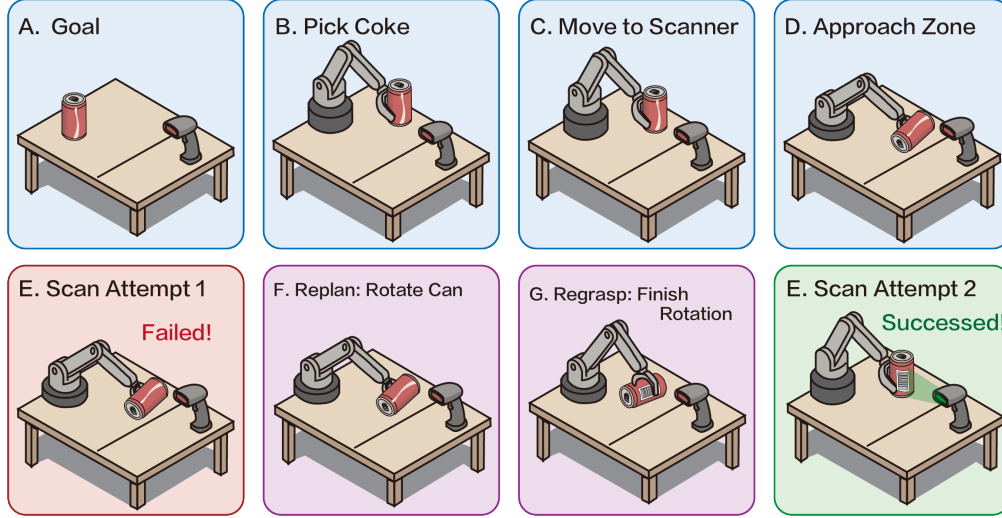


Figure 1 A long-horizon manipulation example requiring memory and reflective replanning. The robot moves a Coke can to a scanner, the first scan fails because the barcode is not visible, and execution feedback triggers a corrective retry. The example highlights the need for persistent task context, intermediate verification, and adaptive recovery beyond reactive action generation.

To address these limitations, recent work has augmented VLA systems with explicit memory mechanisms, including retrieval-based memory, multi-scale representations, and memory-aware reasoning modules (Shi et al., 2025; Sridhar et al., 2025; Torne et al., 2026; Li et al., 2025c). Nevertheless, two fundamental challenges remain. First, memory is often treated as passive storage rather than an active substrate for decision making, limiting its role in verification, diagnosis, and recovery. Second, high-level semantic reasoning and low-level visuomotor control are often entangled within a monolithic architecture, reducing robustness when long-term planning and fine-grained interaction must be handled simultaneously (Yi et al., 2026; Zhang et al., 2026b; Liu et al., 2025).

In this work, we propose a dual-system framework for long-horizon embodied manipulation that explicitly separates *semantic planning* from *visuomotor execution*. The framework consists of a **high-level planner** and a **low-level executor**, inspired by the functional distinction between the cerebrum and cerebellum in biological systems. The high-level planner, instantiated as a vision-language model (VLM), performs *hybrid planning* that combines *static task decomposition* with *dynamic reflective replanning*. Given a global goal and the current observation, it first generates a structured sequence of sub-tasks that specifies semantic instructions, execution conditions, and spatial constraints. Crucially, this plan is not fixed: after each execution step, the planner evaluates post-conditions using memory, verifies whether the intended outcome has been achieved, and performs reflection to diagnose failures. When discrepancies are detected, the planner can either adjust the current sub-task, for example by retrying with updated constraints, or revise the remaining plan. The low-level executor, implemented as a VLA model, translates each sub-task into continuous motor actions through closed-loop visuomotor control.

The central idea is to decouple *memory-guided planning* from *geometry-oriented execution*. The planner maintains a structured memory state that captures task progress, intermediate outcomes, and failure signals, enabling it to determine what to execute next and whether replanning is required. The executor operates on geometry-preserving filtered observations derived from planner-predicted spatial constraints, allowing it to suppress distractors and focus on task-relevant structures (Hung et al., 2025a). In this way, memory is elevated from passive storage to an active component that shapes both high-level decisions and low-level perception.

This design reformulates long-horizon manipulation as a closed-loop process of *planning, execution, verification, and correction*. Instead of assuming successful execution of each sub-task, the system explicitly evaluates post-conditions, identifies potential failure causes, and either retries with updated parameters or triggers dynamic replanning. By structuring the interaction between planning and control, the proposed framework

moves beyond **reactive policies toward a more modular and robust paradigm** for embodied intelligence. Experiments on five representative RMBench tasks show that our framework achieves a 32.4% average success rate, substantially outperforming the strongest baseline at 9.8%. The advantage is especially pronounced on memory-intensive $M(n)$ tasks, where our method reaches 38.7% compared with 9.0% for the best competing approach. Targeted ablations further show that structured memory is a major source of improvement on memory-sensitive tasks and that explicit verification together with reflection significantly improves robustness on failure-prone tasks.

Our contributions are threefold:

- We introduce a dual-system framework that decouples high-level planning from low-level control for long-horizon robotic manipulation.
- We develop a VLM-based planner that unifies task decomposition, memory management, verification, and reflective recovery within a closed-loop decision process.
- We propose a VLA-based executor that performs geometry-oriented action generation under distractor-aware filtered observations, enabling robust execution in complex environments.

2 Related Work

2.1 Vision-Language-Action Foundations

VLA foundation models aim to unify perception, language understanding, and robotic control within a shared Transformer-based policy. This paradigm enables agents to map multimodal observations and natural-language instructions directly to actions. Representative systems such as RT-2 (Zitkovich et al., 2023) and OpenVLA (Kim et al., 2024) demonstrate that large-scale pretraining combined with multi-robot finetuning can substantially enhance instruction following and generalization. Recent advances have further extended VLA capabilities through continuous action generation, diffusion-based control, and large-scale cross-embodiment training, as exemplified by π_0 (Black et al., 2024), RDT-1B (Liu et al., 2024), LingBot-VLA (Wu et al., 2026), and ABot-M0 (Yang et al., 2026).

Despite this progress, existing VLA models still encounter substantial challenges in long-horizon manipulation. These systems must preserve task-relevant context over extended temporal horizons while maintaining stable low-level control under embodiment shifts, real-time constraints, and complex physical interactions (Zitkovich et al., 2023; Kim et al., 2024). These limitations motivate architectures that strengthen temporal organization and task-level reasoning beyond purely reactive action generation.

2.2 Memory-Augmented VLA Systems

Explicit memory is essential for long-horizon embodied tasks because conventional VLA models typically operate within limited observation windows. Existing work has mainly improved long-horizon capability by extending temporal context, strengthening memory retrieval, and coupling memory with higher-level reasoning.

One line of research introduces structured memory representations to preserve information across time. Representative examples include Multi-Scale Embodied Memory (Torne et al., 2026), MemoryVLA (Shi et al., 2025), and ReMem-VLA (Li et al., 2026a), which distinguish recent perceptual context from more abstract long-term task information. Closely related approaches treat memory as an active retrieval interface in which relevant demonstrations or semantic-spatial cues are recalled during inference, as in MAP-VLA (Li et al., 2025b) and Meta-Memory (Mao et al., 2025). Although these methods differ in implementation, they share the goal of making longer-range task context accessible to the policy.

Another direction more tightly couples memory with high-level reasoning and execution feedback. MindExplore (Li et al., 2025a) uses memory-based feedback between reasoning and execution to improve adaptability in long-horizon tasks, whereas NVIDIA ReMEmbR (Anwar et al., 2025) combines vision-language models, LLMs, and retrieval-augmented generation to support persistent embodied deployment. These studies suggest

that memory should contribute not only to context preservation, but also to reasoning about task progress and environmental change.

While recent work has substantially alleviated the temporal-context bottleneck in long-horizon embodied intelligence, two limitations remain. First, memory is often used to preserve or retrieve context, but less often to directly support verification, diagnosis, and corrective action during execution. Second, memory-aware reasoning is often not clearly separated from low-level motor generation, which can reduce robustness when semantic adaptation and fine-grained control must be handled simultaneously. These gaps motivate a modular framework in which structured memory directly supports closed-loop decision making while low-level execution remains specialized and stable.

3 Problem Formulation

Consider a robot operating within an environment defined by state space $\mathcal{S}$ and action space $\mathcal{A}$. At each time step t , the robot receives an observation $o_t = \{I_t, s_t\}$, where $I_t \in \mathcal{O}$ denotes the visual input and $s_t \in \mathbb{R}^{14}$ denotes the proprioceptive state. The action space $\mathcal{A}$ comprises continuous motor commands for two 6-DoF arms and one gripper-control dimension for each arm. A long-horizon manipulation task is specified by a natural-language goal $\mathcal{G}$.

The objective is to maximize the probability of accomplishing $\mathcal{G}$ given the initial observation o_0 . This objective is formalized as a search for an optimal policy π^* that maximizes the expected cumulative success reward over an adaptive sequence of sub-tasks:

$$\pi^* = \arg \max_{\pi} \mathbb{E} \left[\sum_{k=1}^K r_k \mid o_0, \mathcal{G} \right], \quad (1)$$

where K denotes the adaptive number of sub-tasks required for task completion, and $r_k \in \{0, 1\}$ is the binary success reward associated with sub-task τ_k .

Rather than learning a monolithic end-to-end policy that maps observations and language directly to actions, the proposed framework factorizes decision making into two coordinated levels: a high-level planner that generates structured sub-tasks and a low-level executor that performs motor control. Formally, this decomposition is expressed as:

$$\pi^*(a_t \mid o_t, \mathcal{G}) = \underbrace{\pi_{\text{pl}}(\tau_k \mid o_t, \mathcal{G}, \mathcal{M}_t)}_{\text{high-level planner}} \cdot \underbrace{\pi_{\text{ex}}(a_t \mid I_t, s_t, \tau_k)}_{\text{low-level executor}}, \quad (2)$$

where $\mathcal{M}_t$ represents the memory state maintained by the high-level planner and τ_k denotes the current sub-task. Each sub-task is defined as a tuple:

$$\tau_k = (\ell_k, \text{pre}_k, \text{post}_k, \delta_k, \mathcal{B}_k, j_k), \quad (3)$$

where ℓ_k is a natural-language instruction, pre_k and post_k denote the pre-condition and post-condition respectively, δ_k is the maximum execution horizon, $\mathcal{B}_k$ is the set of task-irrelevant spatial constraints, and j_k is the index of the primitive skill selected from the low-level tool library.

Under this formulation, the high-level planner defines the semantic structure of execution by selecting the next sub-task based on the current observation, the global goal, and the accumulated memory. The low-level executor instantiates this decision as continuous control conditioned on the visual observation, the proprioceptive state, and the assigned sub-task. This decomposition decouples long-horizon semantic reasoning from low-level action generation while maintaining closed-loop interaction.

As the low-level executor operates on geometry-preserving filtered observations rather than raw images, the execution policy is expressed as:

$$\pi_{\text{ex}}(a_t \mid I_t, s_t, \tau_k) = \pi_{\text{ex}}(a_t \mid \hat{I}_t, s_t, \ell_k, j_k), \quad \hat{I}_t = \Psi(I_t), \quad (4)$$

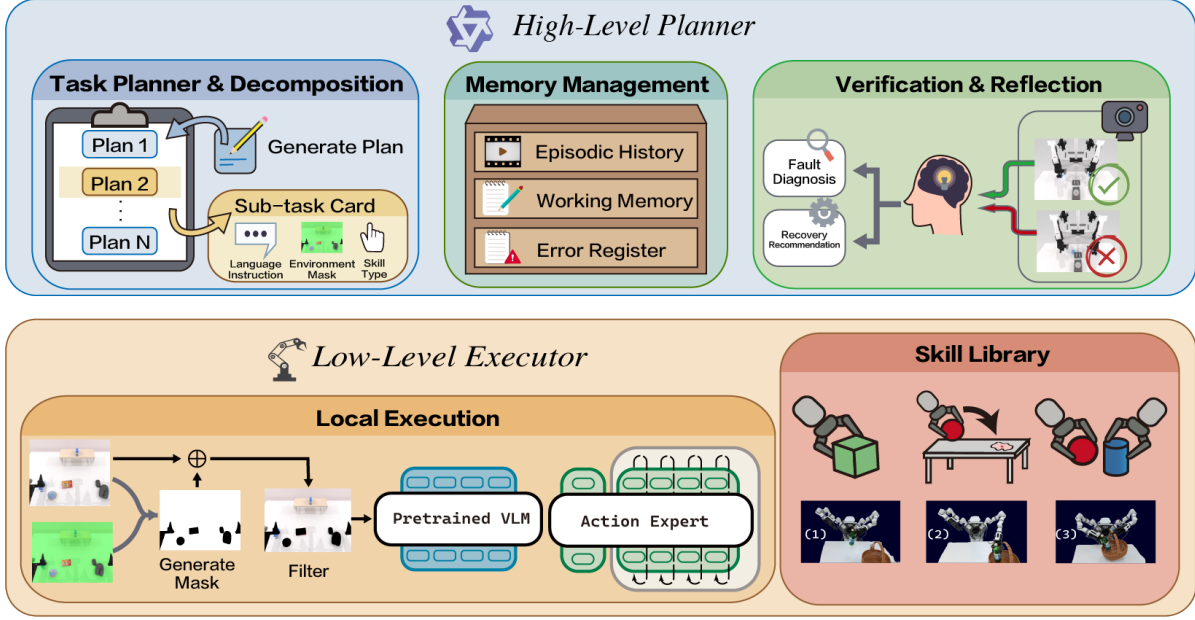


Figure 2 Overview of the proposed dual-system framework. The *high-level planner* manages task planning, memory maintenance, and verification-driven reflection, while the *low-level executor* performs sub-task execution through masked perception and a skill library. This interaction establishes a closed-loop hierarchy for long-horizon embodied manipulation.

where $\hat{I}_t$ denotes the filtered visual observation obtained by suppressing distractor regions specified by $\mathcal{B}_k$. Consequently, the high-level module defines the objective and the associated constraints, whereas the low-level module determines the method of executing the corresponding motor behavior.

The completion of each sub-task is evaluated by a verifier within the high-level planner, which produces:

$$c_k \in \{\text{SUCCESS}, \text{FAIL}, \text{TIMEOUT}\}. \quad (5)$$

This signal determines whether the system proceeds to the subsequent sub-task, retries the current one, or triggers replanning. The framework thus functions as a hierarchical closed-loop policy that coordinates memory-guided semantic planning, geometry-oriented perception, and low-level action generation within a modular implementation.

4 System Architecture

This framework introduces a dual-system approach for long-horizon embodied manipulation by coupling a high-level planner with a low-level executor. The high-level component is implemented as a VLM-based agentic system responsible for planning, memory management, and reflective reasoning. The low-level executor consists of a VLA model that translates individual sub-tasks into continuous motor commands. Figure 2 illustrates the functional division between these systems and the closed-loop interaction that links semantic planning to visuomotor execution.

4.1 Overall Architecture

The framework operates as a closed-loop control hierarchy with two temporal scales, as illustrated in Figure 2. At the slower scale, the high-level planner receives the global goal $\mathcal{G}$ and the current observation o_t , maintains memory $\mathcal{M}_t$, generates or revises a sub-task plan $\mathcal{P}$, detects completion or failure, and triggers reflection when necessary. At the faster scale, the low-level executor receives the current sub-task τ_k and the raw sensor stream, performs geometry-oriented filtering, and generates motor actions $a_t \in \mathcal{A}$ until termination is signaled by the planner.

Interaction between these modules is mediated by a sub-task token τ_k and a completion signal $c_k \in \{\text{SUCCESS}, \text{FAIL}, \text{TIMEOUT}\}$ produced by the high-level planner. The planner defines subsequent objectives together with semantic constraints and execution conditions, while the low-level executor determines how to execute the assigned sub-task through closed-loop visuomotor control. This decomposition decouples long-horizon reasoning from continuous motor execution while maintaining tight coupling through visual feedback and execution outcomes.

4.2 High-Level Planner

The high-level planner is built upon a pre-trained VLM Φ and integrates three functional modules: a task planner, a memory manager, and a reflection engine. These components support long-horizon decision making. Figure 2 illustrates their respective roles within the system architecture, while Figure 3 demonstrates how memory and reflection enable recovery following execution failures.

4.2.1 Task Planning and Decomposition

Given a goal $\mathcal{G}$, an initial observation o_0 , and initial memory $\mathcal{M}_0$, the task planner invokes the VLM to generate an ordered plan:

$$\mathcal{P} = \langle \tau_1, \tau_2, \dots, \tau_K \rangle = \Phi_{\text{plan}}(\mathcal{G}, o_0, \mathcal{M}_0), \quad (6)$$

where Φ_{plan} denotes the VLM prompted to parse the semantic structure of $\mathcal{G}$. This process identifies the pre-conditions and post-conditions for each stage and outputs the sub-tasks as a structured sequence $\mathcal{P}$.

Each sub-task τ_k is defined as a tuple:

$$\tau_k = (\ell_k, \text{pre}_k, \text{post}_k, \delta_k, \mathcal{B}_k, j_k), \quad (7)$$

where ℓ_k is a natural-language instruction, pre_k and post_k are symbolic predicates describing the required and expected world states, δ_k is the maximum execution horizon, $\mathcal{B}_k$ denotes the set of task-irrelevant spatial constraints predicted for the sub-task, and j_k represents the index of the primitive skill selected from the low-level tool library. The planning process is inherently dynamic: the plan may be revised during any decision cycle, resulting in an updated sequence $\mathcal{P}' = \langle \tau_k, \tau'_{k+1}, \dots, \tau'_{K'} \rangle$ to accommodate environmental changes.

The set of spatial constraints is defined as:

$$\mathcal{B}_k = \{b_i\}_{i=1}^{M_k}, \quad b_i = [x_{\min}, y_{\min}, x_{\max}, y_{\max}], \quad (8)$$

where each bounding box identifies a task-irrelevant object or region that may interfere with the current sub-task. Rather than specifying the target directly, these boxes provide top-down cues for scene simplification, guiding the low-level executor to perform geometry-preserving visual filtering prior to motor execution.

4.2.2 Memory Management

The memory module maintains a structured state $\mathcal{M}_t$ to aggregate multimodal context across an entire episode:

$$\mathcal{M}_t = \{ \mathcal{H}_t, \mathcal{W}_t, \mathcal{E}_t \}, \quad (9)$$

where $\mathcal{H}_t$ represents episodic history, $\mathcal{W}_t$ denotes working memory, and $\mathcal{E}_t$ is the error register.

The episodic history $\mathcal{H}_t = \{(o_{t'}, \tau_{k(t')}, a_{t'}, c_{k(t')})\}_{t' < t}$ chronologically records observations, active sub-tasks, actions of the low-level executor, and completion signals. The working memory $\mathcal{W}_t$ is a compact natural-language summary of the world state, which is updated by the VLM following each sub-task transition:

$$\mathcal{W}_{t+1} = \Phi_{\text{mem}}(\mathcal{W}_t, o_t, \tau_k, c_k). \quad (10)$$

The error register stores sub-task failures together with hypothesized causes, as detailed in Section 4.2.3. Figure 3 illustrates how these three components support both execution and recovery.

During each decision cycle, the high-level planner and the reflection engine are conditioned on $\mathcal{M}_t$ to ensure that subsequent decisions incorporate the full task history. To accommodate the finite context window of the

VLM, $\mathcal{H}_t$ is compressed via sliding-window summarization. Only the most recent N_h entries are retained verbatim, while earlier entries are absorbed into $\mathcal{W}_t$.

$$c_k = \Phi_{\text{verify}}\left(o_{t_k^{\text{end}}}^{\text{end}}, \text{post}_k, \mathcal{W}_{t_k^{\text{end}}}\right) \in \{\text{SUCCESS}, \text{FAIL}, \text{TIMEOUT}\}, \quad (11)$$

If $c_k \neq \text{SUCCESS}$, the high-level planner selects a recovery strategy based on the following logic:

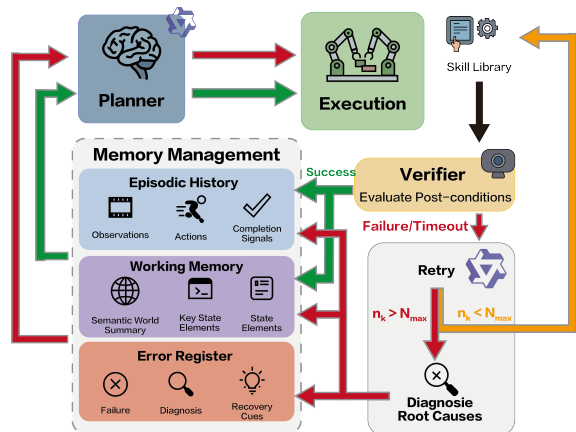
Here, n_k is the count of previous failed attempts for τ_k , and $N_{\max}$ is the retry threshold. The *replan* branch reinvokes the task planner with the updated memory state, producing a revised plan $\mathcal{P}'$ that may substitute, skip, or insert sub-tasks to bypass obstructions.

The reflection engine enables the high-level planner to perform explicit failure analysis when a sub-task is not successfully completed. Upon receiving a failure signal $c_k \neq \text{SUCCESS}$, the engine diagnoses the underlying cause and proposes a corresponding recovery strategy, which is recorded in the error register $\mathcal{E}_t$.

$$(d_k, \rho_k) = \Phi_{\text{reflect}}(\mathcal{F}_k, \mathcal{E}_t), \quad (13)$$

$$\mathcal{E}_{t+1} = \mathcal{E}_t \cup \{(\tau_k, d_k, \rho_k)\}. \quad (14)$$

4.3 Low-Level Executor



At each time step t , the robot obtains an observation $o_t = \{I_t, s_t\}$, where I_t represents the RGB image and $s_t \in \mathbb{R}^{14}$ denotes the proprioceptive state. Given the active sub-task $\tau_k = (\ell_k, \text{pre}_k, \text{post}_k, \delta_k, \mathcal{B}_k, j_k)$, the low-level executor generates motor actions conditioned on the natural-language instruction ℓ_k , the set of distractor constraints $\mathcal{B}_k$, and the selected primitive skill index j_k .

4.3.1 Geometry-Oriented Perception via Masked Filtering

To mitigate visual distractors in cluttered environments, the high-level planner identifies task-irrelevant regions using $\mathcal{B}_k$. The low-level executor subsequently converts these constraints into a geometry-preserving filtered observation. At the onset of a sub-task t_k^{start} , distractor bounding boxes are processed by a zero-shot segmentation model $\mathcal{S}$ to generate a pixel-level mask:

$$Q_{t_k^{\text{start}}}(u, v) = \mathbb{I}[\exists b_i \in \mathcal{B}_k \text{ such that } (u, v) \in \mathcal{S}(I_{t_k^{\text{start}}}, b_i)]. \quad (15)$$

Following initialization, the mask is propagated through time using a lightweight temporal update module:

$$Q_t = \mathcal{K}(I_t, Q_{t-1}), \quad t > t_k^{\text{start}}. \quad (16)$$

The resulting filtered image is computed as

$$\hat{I}_t = \Psi(I_t) = I_t \odot (1 - Q_t), \quad (17)$$

where $\odot$ denotes element-wise multiplication. This operation effectively suppresses distractor regions while preserving the task-relevant geometry of the scene.

4.3.2 Diffusion-Based Skill Library and Action Generation

The low-level executor implements motor behaviors using a discrete library of skills $\mathcal{T} = \{\pi_1, \pi_2, \dots, \pi_J\}$, where each primitive policy is a diffusion-based visuomotor controller specialized for a reusable manipulation task. The skill index j_k selects the active primitive π_{j_k} to be executed. Conditioned on the filtered observation $\hat{I}_t$, the proprioceptive state s_t , and the sub-task instruction embedding ℓ_k , the selected controller generates a sequence of future actions, denoted as an action chunk:

$$\mathbf{A}_t = \{a_t, a_{t+1}, \dots, a_{t+H}\}. \quad (18)$$

Action generation is modeled as a reverse diffusion process:

$$\mathbf{A}_t^{m-1} = \mu_\theta(\mathbf{A}_t^m, m, \hat{I}_t, s_t, \ell_k) + \sigma_m \epsilon, \quad (19)$$

where m represents the diffusion step, μ_θ is a conditional denoising network, and $\sigma_m \epsilon$ denotes the scheduled noise term. The execution policy of the low-level executor is formally expressed as:

$$\pi_{\text{ex}}(\mathbf{A}_t \mid I_t, s_t, \tau_k) = \pi_{j_k}(\mathbf{A}_t \mid \hat{I}_t, s_t, \ell_k), \quad (20)$$

which confirms that the executor operates on the filtered observation $\hat{I}_t = \Psi(I_t)$ rather than the raw visual input I_t .

4.3.3 Local Execution Monitoring and Interface to the High-Level Planner

Given the inherent uncertainty of physical interaction, the low-level executor operates using a receding-horizon approach. Following the execution of each action chunk or at designated checkpoints, the process pauses to return the updated observation to the high-level planner for semantic verification. The low-level executor does not assess global task completion; rather, it provides local progress through the executed action chunk and the latest observation. The high-level planner subsequently determines the completion signal c_k according to Eq. 11. This interface maintains a clear division of labor, where the low-level executor ensures stable geometric control for the current sub-task, while the high-level planner evaluates post-condition satisfaction to decide whether to retry the sub-task or issue a revised plan.

Algorithm 1 summarizes the complete decision cycle of the high-level planner.

Algorithm 1 High-Level Planner Decision Cycle

Require: Goal $\mathcal{G}$, initial observation o_0

```
1: Initialize  $\mathcal{M}_0 \leftarrow \{\emptyset, \emptyset, \emptyset\}$ 
2:  $\mathcal{P} \leftarrow \Phi_{\text{plan}}(\mathcal{G}, o_0, \mathcal{M}_0)$ 
3:  $k \leftarrow 1$ 
4: while  $k \leq |\mathcal{P}|$  and task not complete do
5:   Issue  $\tau_k$  to low-level executor and execute until termination at  $t_k^{\text{end}}$ 
6:    $c_k \leftarrow \Phi_{\text{verify}}(o_{t_k^{\text{end}}}, \text{post}_k, \mathcal{W}_{t_k^{\text{end}}})$ 
7:   Update  $\mathcal{W}_t \leftarrow \Phi_{\text{mem}}(\mathcal{W}_{t-1}, o_t, \tau_k, c_k)$ 
8:   if  $c_k = \text{SUCCESS}$  then
9:      $k \leftarrow k + 1$ 
10:  else
11:     $(d_k, \rho_k) \leftarrow \Phi_{\text{reflect}}(\mathcal{F}_k, \mathcal{E}_t)$ 
12:    Update  $\mathcal{E}_t$ 
13:    if  $\rho_k \in \{\text{RETRY}, \text{ADJUST-PARAM}\}$  and  $n_k < N_{\text{max}}$  then
14:      Optionally modify the parameters of  $\tau_k$  and set  $n_k \leftarrow n_k + 1$ 
15:    else
16:       $\mathcal{P} \leftarrow \Phi_{\text{plan}}(\mathcal{G}, o_t, \mathcal{M}_t)$ 
17:       $k \leftarrow$  index of the next sub-task in the updated  $\mathcal{P}$ 
18:    end if
19:  end if
20: end while
```

5 Experiments

We evaluate our framework from three angles. We first compare it with representative policies on five RMBench tasks that are especially relevant to long-horizon, memory-dependent manipulation. This serves as the main benchmark evaluation. We then test whether the structured memory design improves long-horizon decision making beyond what the policy backbone can do on its own. Finally, we examine whether verification and reflection improve robustness after execution failures by enabling closed-loop recovery. Following this structure, we begin with the benchmark comparison and then present two targeted ablations to clarify which components matter most on memory-sensitive and failure-prone tasks. We also include qualitative rollouts on the same task set to show how the framework maintains task context and completes multi-stage manipulation sequences.

5.1 Experimental Setup

Benchmark and evaluation protocol: Our main quantitative evaluation uses five representative RMBench tasks, which are also used in the targeted analyses: two M(1) tasks (Observe and Pick Up, Rearrange Blocks) and three M(n) tasks (Battery Try, Blocks Ranking Try, Press Button). For our method, we follow the RMBench protocol and train with 50 expert demonstrations per task, and we evaluate over 100 rollout episodes. We use task success rate as the primary metric and also report the average success rate on the M(1) subset, the M(n) subset, and the full task set.

Training details: Our method is trained for 30k optimization steps using demonstrations decomposed into preprocessed subtasks. This gives subtask-level supervision for the hierarchical execution pipeline and matches the structure of the proposed framework. Unless noted otherwise, we report results from the final checkpoint. The baseline numbers in the main comparison are taken directly from RMBench under the same evaluation protocol.

Ablation protocol: To keep the mechanism analysis aligned with the benchmark comparison, we conduct both ablations on the same task pool while splitting the tasks by the component under study. The memory ablation uses Observe and Pick Up, Rearrange Blocks, and Blocks Ranking Try, which require target retention, intermediate state tracking, and multi-stage bookkeeping. The recovery ablation uses Battery Try and

Table 1 Comparison with baselines on RMBench (Chen et al., 2026). We report success rate (%) on five representative benchmark tasks. The baseline values are taken from RMBench, and the results of our method are measured under the same protocol.

Method	M(1) Tasks		Battery Try	M(n) Tasks		Average		
	Observe and Pick Up	Rearrange Blocks		Blocks Ranking Try	Press Button	M(1)	M(n)	Total
DP (Chi et al., 2025)	1%	0%	10%	10%	0%	0.5%	6.7%	4.2%
ACT (Zhao et al., 2023)	1%	29%	19%	0%	0%	15.0%	6.3%	9.8%
Pi0.5 (Black et al., 2024)	9%	13%	16%	6%	0%	11.0%	7.3%	8.8%
X-VLA (Zheng et al., 2025)	9%	13%	26%	1%	0%	11.0%	9.0%	9.8%
Ours	8%	38%	46%	60%	10%	23.0%	38.7%	32.4%

Press Button, which are more sensitive to execution failure and therefore better suited to evaluating explicit verification and reflective correction. All ablation variants use the same policy backbone, training budget, and evaluation protocol as the full model, and differ only in the modules included for analysis. For both ablations, we report success rate over 100 evaluation episodes (SR@100).

Qualitative evaluation: In addition to average success rates, we visualize representative rollouts on the same five benchmark tasks to examine whether the framework preserves task-relevant context throughout execution. These examples complement the numerical results by showing the temporal structure of the manipulation process rather than only the final binary outcome.

5.2 Comparison with Baselines

We compare our framework with four representative baselines: DP (Chi et al., 2025), ACT (Zhao et al., 2023), Pi0.5 (Black et al., 2024), and X-VLA (Zheng et al., 2025). DP and ACT are non-pretrained policies, whereas Pi0.5 and X-VLA are pretrained VLA-style policies. The main quantitative results are shown in Table 1. We group the reported tasks by memory complexity because the key question is not only whether a method achieves a higher overall average, but also whether it remains reliable as task dependencies become longer and more stateful. In this task subset, the M(1) tasks mainly test short-range retention or local state dependence, whereas the M(n) tasks place much heavier demands on persistent task context, sequential bookkeeping, and delayed consequences across multiple stages.

Table 1 first shows that the selected RMBench subset is difficult for all baselines. Even the strongest competing methods remain below 10% in total average, and their performance usually drops when moving from M(1) to M(n). Taken together, these results suggest that stronger action priors alone are not enough to address the main challenge in this setting, namely maintaining task-relevant context across multiple dependent stages.

Against this background, our framework shows a clear overall advantage. It reaches 32.4% in total average, far above the best baseline result of 9.8%. This improvement is not confined to one or two easy tasks. The method remains competitive on the M(1) subset and improves much more strongly on the M(n) subset, where long-horizon dependencies are more pronounced. Specifically, it achieves 23.0% on M(1) and 38.7% on M(n), whereas the strongest baseline averages are 15.0% and 9.0%, respectively. The larger margin on M(n) is consistent with the intended role of the framework.

The task-level results help explain where this gain comes from. On Rearrange Blocks, our framework reaches



Figure 4 Qualitative results of our framework on five representative RMBench tasks. From top to bottom, the task sequences correspond to Observe and Pick Up, Rearrange Blocks, Battery Try, Blocks Ranking Try, and Press Button. Each row shows key frames from one rollout and illustrates that the method can complete representative memory-dependent manipulation tasks with different sequential interaction requirements.

38%, improving over the strongest baseline at 29%. On Battery Try, it reaches 46%, compared with the best baseline result of 26%. The largest margin appears on Blocks Ranking Try, where our framework achieves 60% while all baselines remain at or below 10%. On Press Button, it is the only method with non-zero success. These results point to a consistent pattern: the advantage is most visible when success depends on preserving intermediate state, coordinating dependent interactions, or returning to earlier task context after additional actions.

At the same time, the method is not uniformly better on every task. On Observe and Pick Up, it reaches 8%, slightly below the best baseline value of 9%. This makes the overall trend more informative. The main gain does not come from uniformly stronger short-horizon execution, but from much stronger performance on tasks whose difficulty is more tightly tied to persistent memory, multi-stage reasoning, and long-horizon coordination.

Figure 4 complements the quantitative comparison by showing representative rollouts on the same five tasks. The examples show coherent progression across observation, rearrangement, trial-based interaction, ranking, and final goal verification. This visual evidence is consistent with the numerical results in Table 1: the framework improves not only the final success statistic, but also the stability of task progression throughout the rollout.

Overall, the main benchmark results support two conclusions. The first is that our framework is clearly competitive on the selected RMBench subset under the standard protocol. The second, and more important one, is that its advantage becomes larger as the tasks place heavier demands on long-horizon memory and sequential dependency handling. This trend directly motivates the two targeted ablations below.

5.3 Targeted Ablations

The benchmark comparison establishes the effectiveness of the full system, but it does not by itself explain which components are responsible for the improvement. We therefore conduct two targeted ablations, one focused on structured memory and the other on recovery after failed execution. To keep both analyses directly comparable with the main benchmark, the ablations are carried out on tasks drawn from Table 1, with each task subset chosen to match the mechanism under study.

We begin with the memory ablation in Table 2. The base variant performs poorly on all three tasks, with an average of 6.7%, showing that the backbone alone is not sufficient when the task requires delayed target recall, intermediate state preservation, or longer sequential bookkeeping. Adding episodic history changes this picture immediately: the average rises to 27.7%, and the gain is especially large on Blocks Ranking Try, which increases from 6% to 54%. This suggests that retaining temporally ordered task experience already provides a strong basis for long-horizon behavior.

The next comparison is more nuanced. When working memory is added on top of episodic history, the average changes only slightly, from 27.7% to 28.0%, but the effect differs across tasks. Rearrange Blocks improves substantially, rising from 21% to 36%, which suggests that a compact state summary is useful when the agent must track an evolving spatial arrangement. In contrast, Observe and Pick Up decreases from 8% to 6%, and Blocks Ranking Try decreases from 54% to 42%. This suggests that working memory is not a uniform gain term across tasks. Its contribution depends on the structure of the task and on whether the current decision benefits more from abstraction or from direct access to accumulated experience.

The full model achieves the best overall result, reaching 35.3% on average and improving all three tasks relative to the base model. In particular, it raises Rearrange Blocks to 38% and Blocks Ranking Try to 60%, while restoring Observe and Pick Up to 8%. Taken together, these results suggest that the benefit of structured memory does not come from a single component alone. Episodic history provides the first major improvement, while the full memory design works best when its components operate together within the complete decision pipeline.

Table 3 then turns to the second claim of the paper, namely whether explicit checking and reflection improve robustness after failed execution. Here the trend is more monotonic. Starting from the base model, the average success rate is 8.0%. Adding the verifier raises it to 17.5%, showing that explicit post-condition checking already improves robustness by detecting incomplete or failed outcomes that would otherwise go

Table 2 Memory ablation study. We evaluate whether structured memory improves performance on benchmark tasks that require target retention, intermediate state tracking, or multi-stage bookkeeping.

Setting	Observe and Pick Up	Rearrange Blocks	Blocks Ranking Try	Avg.
Base	4%	10%	6%	6.7%
+ $\mathcal{H}_t$	8%	21%	54%	27.7%
+ $\mathcal{H}_t + \mathcal{W}_t$	6%	36%	42%	28.0%
Full model	8%	38%	60%	35.3%

Table 3 Recovery ablation study. We evaluate whether verification and reflection improve robustness on benchmark tasks that are sensitive to failed execution and corrective retry.

Setting	Battery Try	Press Button	Avg.
Base	16%	0%	8.0%
+ Φ_{verify}	30%	5%	17.5%
+ $\Phi_{\text{verify}} + \Phi_{\text{reflect}}$	38%	10%	24.0%
Full model	46%	10%	28.0%

unnoticed. This gain appears on both tasks, with Battery Try improving from 16% to 30% and Press Button improving from 0% to 5%.

Adding reflection on top of verification yields a further increase to 24.0%. The improvement is again visible on both tasks and is especially clear on Battery Try, which rises to 38%. This suggests that detecting failure is helpful, but not sufficient on its own. Once a failure has been identified, the system still needs a mechanism that turns that signal into a corrective decision, such as retrying, adjusting execution, or replanning. The role of reflection is therefore not simply to recognize failure, but to provide a more actionable response for recovery.

The full model reaches 28.0%, the strongest average among all recovery variants, and increases Battery Try further to 46% while maintaining 10% on Press Button. Although the gain from the last step is smaller than the gain from introducing verification, the overall trend remains consistent across the ablation: robustness improves as the recovery loop becomes more explicit and more complete. This is in line with the intended design of the framework, in which verification determines whether execution has succeeded and reflection determines how the system should respond when it has not.

Overall, the experiments support a coherent picture. The main RMBench comparison shows that our framework is clearly stronger than representative baselines on the selected long-horizon task subset, with the largest gains appearing on the more memory-intensive $M(n)$ tasks. The memory ablation shows that persistent task state is a major source of this improvement, especially when the task requires long sequential bookkeeping. The recovery ablation further shows that explicit verification and reflection improve robustness on tasks where failed execution must be detected and corrected before the rollout can succeed. Taken together, these results support the central claim of the paper: long-horizon manipulation benefits not only from stronger action generation, but also from memory-guided decision making and closed-loop correction.

6 Conclusion

We presented a dual-system framework for long-horizon embodied manipulation that explicitly separates high-level semantic planning from low-level visuomotor execution. The proposed framework combines a VLM-based planner with structured memory, outcome verification, and reflection-driven recovery, together with a VLA-based executor that performs geometry-oriented action generation on filtered observations. This design reformulates long-horizon manipulation as a closed-loop process of planning, execution, verification, and correction, rather than relying solely on reactive end-to-end action prediction.

Experiments on five representative RMBench tasks showed that the proposed framework substantially outperforms representative baselines, with especially large gains on memory-intensive $M(n)$ tasks. Targeted ablations further demonstrated that structured memory is a major source of improvement for long-horizon decision making, while explicit verification and reflection improve robustness on failure-prone tasks. These

results suggest that effective long-horizon embodied manipulation requires not only stronger action generation, but also modular coordination among persistent task memory, semantic reasoning, and adaptive recovery.

In future work, we plan to extend this framework to broader real-world manipulation settings, strengthen the interaction between memory updating and low-level adaptation, and study how the same closed-loop design can support more diverse long-horizon tasks under stronger scene uncertainty and embodiment variation.

References

- Abrar Anwar, John Welsh, Joydeep Biswas, Soha Pouya, and Yan Chang. Remembr: Building and reasoning over long-horizon spatio-temporal memory for robot navigation. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 2838–2845. IEEE, 2025.
- K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, et al. π_0 : A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.
- Tianxing Chen, Yuran Wang, Mingleyang Li, Yan Qin, Hao Shi, Zixuan Li, Yifan Hu, Yingsheng Zhang, Kaixuan Wang, Yue Chen, et al. Rmbench: Memory-dependent robotic manipulation benchmark with insights into policy design. *arXiv preprint arXiv:2603.01229*, 2026.
- Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, 44(10-11):1684–1704, 2025.
- Shengliang Deng, Mi Yan, Songlin Wei, Haixin Ma, Yuxin Yang, Jiayi Chen, Zhiqi Zhang, Taoyu Yang, Xuheng Zhang, Wenhao Zhang, et al. Graspvla: a grasping foundation model pre-trained on billion-scale synthetic action data. *arXiv preprint arXiv:2505.03233*, 2025.
- Muhayy Ud Din, Waseem Akram, Lyes Saad Saoud, Jan Rosell, and Irfan Hussain. Vision language action models in robotic manipulation: A systematic review. *arXiv preprint arXiv:2507.10672*, 2025.
- Zhe Hu, Jing Li, Zhongzhu Pu, Hou Pong Chan, and Yu Yin. Praxis-VLM: Vision-grounded decision making via text-driven reinforcement learning. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=U806q3iILo>.
- Chi-Pin Huang, Yueh-Hua Wu, Min-Hung Chen, Yu-Chiang Frank Wang, and Fu-En Yang. Thinkact: Vision-language-action reasoning via reinforced visual latent planning. *arXiv preprint arXiv:2507.16815*, 2025.
- Chia-Yu Hung, Navonil Majumder, Haoyuan Deng, Liu Renhang, Yankang Ang, Amir Zadeh, Chuan Li, Dorien Herremans, Ziwei Wang, and Soujanya Poria. Nora-1.5: A vision-language-action model trained using world model-and action-based preference rewards. *arXiv preprint arXiv:2511.14659*, 2025a.
- Chia-Yu Hung, Qi Sun, Pengfei Hong, Amir Zadeh, Chuan Li, U Tan, Navonil Majumder, Soujanya Poria, et al. Nora: A small open-sourced generalist vision language action model for embodied tasks. *arXiv preprint arXiv:2504.19854*, 2025b.
- Sicong Jiang, Zilin Huang, Kangan Qian, Ziang Luo, Tianze Zhu, Yang Zhong, Yihong Tang, Menglin Kong, Yunlong Wang, Siwen Jiao, et al. A survey on vision-language-action models for autonomous driving. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 4524–4536, 2025.
- Kento Kawaharazuka, Jihoon Oh, Jun Yamada, Ingmar Posner, and Yuke Zhu. Vision-language-action models for robotics: A review towards real-world applications. *IEEE Access*, 2025.
- Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, et al. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.
- Daixun Li, Yusi Zhang, Mingxiang Cao, Donglai Liu, Weiying Xie, Tianlin Hui, Lunkai Lin, Zhiqiang Xie, and Yunsong Li. Towards long-horizon vision-language-action system: Reasoning, acting and memory. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 6839–6848, 2025a.
- Hang Li, Fengyi Shen, Dong Chen, Liudi Yang, Xudong Wang, Jinkui Shi, Zhenshan Bing, Ziyuan Liu, and Alois Knoll. Remem-vla: Empowering vision-language-action model with memory via dual-level recurrent queries. *arXiv preprint arXiv:2603.12942*, 2026a.
- Ruiying Li, Yunlang Zhou, YuYao Zhu, Kylin Chen, Jingyuan Wang, Sukai Wang, Kongtao Hu, Minhui Yu, Bowen Jiang, Zhan Su, et al. Roboclaw: An agentic framework for scalable long-horizon robotic tasks. *arXiv preprint arXiv:2603.11558*, 2026b.
- Runhao Li, Wenkai Guo, Zhenyu Wu, Changyuan Wang, Haoyuan Deng, Zhenyu Weng, Yap-Peng Tan, and Ziwei Wang. Map-vla: Memory-augmented prompting for vision-language-action model in robotic manipulation. *arXiv preprint arXiv:2511.09516*, 2025b.

- Zhiyu Li, Shichao Song, Hanyu Wang, Simin Niu, Ding Chen, Jiawei Yang, Chenyang Xi, Huayi Lai, Jihao Zhao, Yezhaohui Wang, et al. Memos: An operating system for memory-augmented generation (mag) in large language models. *arXiv preprint arXiv:2505.22101*, 2025c.
- Shijie Lian, Bin Yu, Xiaopeng Lin, Laurence T Yang, Zhaolong Shen, Changti Wu, Yuzhuo Miao, Cong Huang, and Kai Chen. Langforce: Bayesian decomposition of vision language action models via latent action queries. *arXiv e-prints*, pp. arXiv-2601, 2026.
- Jiaming Liu, Hao Chen, Pengju An, Zhuoyang Liu, Renrui Zhang, Chenyang Gu, Xiaoqi Li, Ziyu Guo, Sixiang Chen, Mengzhen Liu, et al. Hybridvla: Collaborative diffusion and autoregression in a unified vision-language-action model. *arXiv preprint arXiv:2503.10631*, 2025.
- Songming Liu, Lingxuan Wu, Bangguo Li, Hengkai Tan, Huayu Chen, Zhengyi Wang, Ke Xu, Hang Su, and Jun Zhu. Rdt-1b: a diffusion foundation model for bimanual manipulation. *arXiv preprint arXiv:2410.07864*, 2024.
- Yueen Ma, Zixing Song, Yuzheng Zhuang, Jianye Hao, and Irwin King. A survey on vision-language-action models for embodied ai. *arXiv preprint arXiv:2405.14093*, 2024.
- Yufan Mao, Hanjing Ye, Wenlong Dong, Chengjie Zhang, and Hong Zhang. Meta-memory: Retrieving and integrating semantic-spatial memories for robot spatial reasoning. *arXiv preprint arXiv:2509.20754*, 2025.
- Hao Shi, Bin Xie, Yingfei Liu, Lin Sun, Fengrong Liu, Tiancai Wang, Erjin Zhou, Haoqiang Fan, Xiangyu Zhang, and Gao Huang. Memoryvla: Perceptual-cognitive memory in vision-language-action models for robotic manipulation. *arXiv preprint arXiv:2508.19236*, 2025.
- Ajay Sridhar, Jennifer Pan, Satvik Sharma, and Chelsea Finn. Memer: Scaling up memory for robot control via experience retrieval. *arXiv preprint arXiv:2510.20328*, 2025.
- GigaBrain Team, Boyuan Wang, Chaojun Ni, Guan Huang, Guosheng Zhao, Hao Li, Jie Li, Jindi Lv, Jingyu Liu, Lv Feng, et al. Gigabrain-0.5 m*: a vla that learns from world model-based reinforcement learning. *arXiv preprint arXiv:2602.12099*, 2026.
- Spirit AI Team. Spirit-v1.5: Clean data is the enemy of great robot foundation models. *Spirit AI Blog*, 2026. <https://www.spirit-ai.com/en/blog/spirit-v1-5>.
- Marcel Torne, Karl Pertsch, Homer Walke, Kyle Vedder, Suraj Nair, Brian Ichter, Allen Z Ren, Haohuan Wang, Jiaming Tang, Kyle Stachowicz, et al. Mem: Multi-scale embodied memory for vision language action models. *arXiv preprint arXiv:2603.03596*, 2026.
- Wei Wu, Fan Lu, Yunnan Wang, Shuai Yang, Shi Liu, Fangjing Wang, Qian Zhu, He Sun, Yong Wang, Shuailei Ma, et al. A pragmatic vla foundation model. *arXiv preprint arXiv:2601.18692*, 2026.
- Tian-Yu Xiang, Ao-Qun Jin, Xiao-Hu Zhou, Mei-Jiang Gui, Xiao-Liang Xie, Shi-Qi Liu, Shuang-Yi Wang, Sheng-Bin Duan, Fu-Chao Xie, Wen-Kai Wang, et al. Parallels between vla model post-training and human motor learning: Progress, challenges, and trends. *arXiv preprint arXiv:2506.20966*, 2025.
- Yandan Yang, Shuang Zeng, Tong Lin, Xinyuan Chang, Dekang Qi, Junjin Xiao, Haoyun Liu, Ronghan Chen, Yuzhi Chen, Dongjie Huo, et al. Abot-m0: Vla foundation model for robotic manipulation with action manifold learning. *arXiv preprint arXiv:2602.11236*, 2026.
- Seonghyeon Ye, Yunhao Ge, Kaiyuan Zheng, Shenyuan Gao, Sihyun Yu, George Kurian, Suneel Indupuru, You Liang Tan, Chuning Zhu, Jiannan Xiang, et al. World action models are zero-shot policies. *arXiv preprint arXiv:2602.15922*, 2026.
- Pengfei Yi, Yingjie Ma, Wenjiang Xu, Yanan Hao, Shuai Gan, Wanting Li, and Shanlin Zhong. Critic in the loop: A tri-system vla framework for robust long-horizon manipulation. *arXiv preprint arXiv:2603.05185*, 2026.
- En Yu, Haoran Lv, Jianjian Sun, Kangheng Lin, Ruitao Zhang, Yukang Shi, Yuyang Chen, Ze Chen, Ziheng Zhang, Fan Jia, et al. Dm0: An embodied-native vision-language-action model towards physical ai. *arXiv preprint arXiv:2602.14974*, 2026.
- Yang Yue, Yulin Wang, Bingyi Kang, Yizeng Han, Shenzhi Wang, Shiji Song, Jiashi Feng, and Gao Huang. Deer-vla: Dynamic inference of multimodal large language models for efficient robot execution. *Advances in Neural Information Processing Systems*, 37:56619–56643, 2024.
- Jianke Zhang, Xiaoyu Chen, Qiuyue Wang, Mingsheng Li, Yanjiang Guo, Yucheng Hu, Jiajun Zhang, Shuai Bai, Junyang Lin, and Jianyu Chen. Vlm4vla: Revisiting vision-language-models in vision-language-action models. *arXiv preprint arXiv:2601.03309*, 2026a.

- Rongyu Zhang, Menghang Dong, Yuan Zhang, Liang Heng, Xiaowei Chi, Gaole Dai, Li Du, Dan Wang, Yuan Du, and Shanghang Zhang. Mole-vla: Dynamic layer-skipping vision language action model via mixture-of-layers for efficient robot manipulation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pp. 18764–18772, 2026b.
- Tony Z Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. *arXiv preprint arXiv:2304.13705*, 2023.
- Jinliang Zheng, Jianxiong Li, Zhihao Wang, Dongxiu Liu, Xirui Kang, Yuchun Feng, Yinan Zheng, Jiayin Zou, Yilun Chen, Jia Zeng, et al. X-vla: Soft-prompted transformer as scalable cross-embodiment vision-language-action model. *arXiv preprint arXiv:2510.10274*, 2025.
- Brianna Zitkovich, Tianhe Yu, Sichun Xu, Peng Xu, Ted Xiao, Fei Xia, Jialin Wu, Paul Wohlhart, Stefan Welker, Ayzaan Wahid, et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning*, pp. 2165–2183. PMLR, 2023.